

Introducción a Apache Hadoop - Guía Completa

1. Motivación, Origen e Historia

Contexto Inicial (2002-2006)

- **Problema:** Doug Cutting y Mike Cafarella trabajaban en **Apache Nutch** (motor de búsqueda) que necesitaba indexar **mil millones de páginas web**.
- **Limitación tecnológica:** Con la tecnología del momento, el coste estimado era de **medio millón de euros en hardware + medio millón anual en mantenimiento**.
- **Inspiración en Google:**
- **2003:** Google publica el paper "**The Google File System**" (sistema de almacenamiento distribuido).
- **2004:** Google publica "**MapReduce: Simplified Data Processing on Large Clusters**" (paradigma de procesamiento distribuido).
- **Nacimiento de Hadoop:** En **2006**, Doug Cutting separa estas implementaciones de Nutch y crea **Hadoop** como proyecto independiente.
- **Curiosidad:** El nombre proviene del **peluche elefante** de su hijo.

Evolución Clave

- **2007:** Primera versión beta en Yahoo! (probada en 1000+ nodos).
 - **2008:** Yahoo! dona Hadoop a la **Apache Software Foundation**.
 - **2009:** Doug Cutting se une a **Cloudera** para hacer Hadoop empresarial.
 - **2011:** Liberación de **Hadoop 1.0** (primera versión estable).
-

2. ¿Qué es Apache Hadoop?

Definición Integral

Apache Hadoop es una **plataforma open-source** que permite **almacenar y procesar grandes volúmenes de datos** a bajo coste, con las siguientes características:

Característica	Explicación	Ejemplo/Implicación
Plataforma	Base para construir aplicaciones (no solo una herramienta).	Similar a una "caja de herramientas" para Big Data.
Open-source	Sin costes de licencia.	Código libre accesible para todos.
Almacenamiento	Mediante HDFS (Hadoop Distributed File System).	Almacena petabytes de datos distribuidos.
Procesamiento	Batch (MapReduce) y real-time (YARN).	Análisis de logs GPS para optimizar rutas de transporte.
Bajo coste	Coste órdenes de magnitud menor vs. bases de datos relacionales.	Escalabilidad lineal: añadir capacidad cuesta proporcionalmente.
Volumen	Desde MB hasta PB.	Caso práctico: datos de GPS de flota de camiones.
Estructura	Acepta datos estructurados, semiestructurados y no estructurados .	Vídeos, logs, imágenes, JSON, etc.
Distribuido	Múltiples servidores (nodos) trabajando conjuntamente.	Cluster de decenas a miles de nodos.
Escalabilidad horizontal	Añadir más nodos similares (no servidores más grandes).	Límite teórico ~10,000 nodos.

Característica	Explicación	Ejemplo/Implicación
Tolerante a fallos	Soporta caídas de nodos sin perder datos.	En cluster de 1,000 nodos, con 12 discos/nodo, pueden fallar ~11 discos/día sin impacto.
Hardware commodity	No requiere hardware especializado.	Servidores estándar con discos HDD/SSD.
Procesamiento cercano a datos	Minimiza transferencia por red (evita cuellos de botella).	El código se ejecuta donde están los datos.

Comparativa con Sistemas Tradicionales

- **Mainframes y bases de datos relacionales** (Oracle, DB2) ya manejaban grandes volúmenes antes de Hadoop.
- **Diferencia clave:** Hadoop reduce costes **10x a 1000x** gracias a hardware commodity y modelo distribuido.

3. Ecosistema Hadoop y Distribuciones

Componentes Core

1. **HDFS (Hadoop Distributed File System):** - Sistema de archivos distribuido (almacenamiento). - Estructura jerárquica (directorios/subdirectorios).
2. **YARN (Yet Another Resource Negotiator):** - Gestor de recursos y procesamiento. - Permite ejecutar aplicaciones sobre HDFS.
3. **MapReduce:** - Paradigma de procesamiento batch. - API para programación distribuida.

Proyectos del Ecosistema Apache

Proyecto	Función	Lenguaje/ Interfaz
Apache Hive	Acceso a datos en HDFS como tablas relacionales.	SQL-like (HiveQL)
Apache Pig	Flujos de transformación de datos.	Lenguaje de scripting (Pig Latin)
Apache HBase	Base de datos NoSQL columnar.	API Java/REST
Otros comunes: Spark, Kafka, Sqoop, Flume, Oozie, etc.		

Distribuciones Empresariales

- **Cloudera CDH, Hortonworks HDP, MapR:** Empaquetan componentes + herramientas de gestión/monitorización.
- **Objetivo:** Facilitar instalación, operación y soporte para entornos productivos.

4. Arquitectura de Hadoop

Modelo de Despliegue

- **Cluster:** Conjunto de nodos (servidores) conectados en red.
- **Tipos de nodos:**
 - **Nodo Maestro (Master):** Gestiona metadata y coordinación (NameNode, ResourceManager).
 - **Nodos Esclavos (Workers):** Almacenan datos y ejecutan procesamiento (DataNodes, NodeManagers).

Principio de Diseño Clave

- **"Move computation, not data":** Enviar el código a los nodos donde están los datos (vs. sistemas tradicionales que mueven datos a través de la red).
-

5. Beneficios, Desventajas y Dificultades

Beneficios

- **Coste reducido** (hardware commodity, open-source).
- **Escalabilidad horizontal** casi ilimitada.
- **Tolerancia a fallos** automática.
- **Flexibilidad** en tipos de datos y cargas de trabajo.

Desventajas/Dificultades

- **Complejidad:** Múltiples componentes requieren expertise.
 - **Latencia:** No optimizado para consultas en tiempo real muy bajas (<1s).
 - **Administración:** Clusters grandes necesitan operación especializada.
 - **Seguridad:** Configuración no trivial (Kerberos, cifrado).
 - **Madurez:** En versiones tempranas, falta de herramientas de gestión empresarial.
-

Resumen Ejecutivo (100 palabras)

Apache Hadoop nació en 2006 para resolver el almacenamiento y procesamiento económico de grandes volúmenes de datos, inspirado en los papers de Google File System y MapReduce. Es una plataforma open-source distribuida que escala horizontalmente usando hardware estándar, es tolerante a fallos y procesa datos cerca de donde se almacenan. Su ecosistema incluye herramientas como Hive, Pig y HBase. Aunque reduce costes drásticamente frente a soluciones tradicionales, presenta complejidad operativa. Hadoop democratizó el Big Data al hacer accesible el procesamiento masivo a cualquier organización.

Ejemplo de Aplicación Práctica: Una empresa de transporte usa Hadoop para almacenar datos GPS de su flota (HDFS) y ejecutar análisis con MapReduce para optimizar rutas, detectar malas prácticas de conducción y predecir mantenimientos predictivos, todo a un coste asumible.

Resumen Detallado: Ecosistema y Arquitectura de Hadoop

Parte 1: El Ecosistema Apache Hadoop

Componentes Principales del Ecosistema

El ecosistema Hadoop está compuesto por múltiples proyectos independientes de la Apache Software Foundation, cada uno con una función específica. Los más relevantes son:

- **Componentes Core (Fundamentales):**

- **HDFS (Hadoop Distributed File System):** Sistema de archivos distribuido y tolerante a fallos.
- **YARN (Yet Another Resource Negotiator):** Gestor de recursos y planificador de tareas para el clúster.

- **Componentes de Ingesta y Movimiento de Datos:**

- **Apache Sqoop:** Importa/exporta datos estructurados entre bases de datos relacionales (como Oracle, MySQL) y Hadoop.
- **Apache Flume:** Ingesta streams de datos en tiempo real (logs, eventos) hacia HDFS.
- **Apache Kafka:** Sistema de mensajería para recolección y procesamiento de eventos en tiempo real a gran escala.

- **Componentes de Procesamiento y Consulta:**

- **Apache Spark:** Motor de procesamiento masivo muy eficiente para ingeniería de datos, machine learning y procesamiento de grafos. Es el complemento más popular de Hadoop actualmente.
- **Apache Hive:** Permite consultar datos en HDFS usando un lenguaje similar a SQL (HiveQL).
- **Apache Impala:** Ofrece funcionalidad similar a Hive pero con un rendimiento más elevado y tiempos de respuesta menores para consultas interactivas.
- **Apache Storm:** Sistema de procesamiento de eventos en tiempo real con baja latencia.

- **Componentes de Almacenamiento y Acceso:**

- **Apache HBase:** Base de datos NoSQL de tipo columnar que permite acceso aleatorio y atómico a datos en HDFS.
- **Apache Phoenix:** Capa que permite acceder a los datos de HBase mediante una interfaz SQL.

- **Componentes de Orquestación, Gobierno y Utilidades:**

- **Apache Oozie:** Herramienta para definir, orquestar y planificar flujos de trabajo (workflows) en Hadoop.
- **Apache ZooKeeper:** Servicio de coordinación para sincronizar el estado de los servicios distribuidos del clúster.
- **Apache Atlas:** Herramienta de gobierno y metadata para catalogar y rastrear el linaje de los datos.
- **Apache Zeppelin:** Aplicación web (notebook) que facilita el análisis de datos, la colaboración y la visualización para científicos de datos.

Recomendación Práctica: No es necesario dominar todos los componentes. En proyectos reales se suele usar un subconjunto según las necesidades. Los más utilizados son **Apache Spark, Apache Hive y Apache Kafka**, además de los componentes core (HDFS y YARN).

El Desafío de las Dependencias y el Soporte

Cada componente Apache tiene su propio ciclo de versiones, dependencias y roadmap, lo que hace muy complejo: 1. **Instalar una plataforma "Vanilla" (desde cero):** Gestionar manualmente las dependencias entre versiones de

componentes (ej: la versión X de Phoenix requiere la versión Y de HBase) es difícil y propenso a errores. 2. **Resolver incidencias en producción:** Diagnosticar un fallo implica investigar en múltiples componentes interconectados, lo que ralentiza la resolución.

Soluciones: Distribuciones Comerciales y Cloud

Para superar estos desafíos, surgieron alternativas que empaquetan y simplifican el ecosistema:

1. Distribuciones Comerciales (On-Premise):

- Ofrecen un instalador unificado, resuelven dependencias, incluyen utilidades adicionales y proporcionan **soporte empresarial 24x7**.
- **Evolución del mercado:** Tras fusiones y adquisiciones, **Cloudera** es actualmente la principal y casi única distribución comercial, integrando las funcionalidades de la desaparecida Hortonworks.

2. Hadoop-as-a-Service en la Nube (Cloud):

- Servicios gestionados por proveedores cloud que permiten desplegar clústeres Hadoop de forma elástica en minutos, con un modelo de **pago por uso**.
- **Principales ofertas:** Amazon EMR, Microsoft Azure HDInsight y Google Dataproc.
- **Ventajas clave:**
 - **Reducción del Time-to-Market:** Aprovisionamiento en minutos vs. meses on-premise.
 - **Elasticidad:** Escalar la capacidad del clúster (arriba o abajo) es sencillo.
 - **Reducción de riesgo:** No requiere gran inversión inicial en hardware y licencias.
- **Desventajas:**
 - **Vendor Lock-in:** Dificultad y coste para migrar a otro proveedor o volver a on-premise.
 - **Costes impredecibles:** Las fórmulas de cálculo pueden incluir variables difíciles de estimar.
 - **Soluciones no estándar:** Los proveedores suelen usar adaptaciones propias de los componentes Apache.

Conclusión: La competencia principal para Cloudera ya no son otras distribuciones, sino los grandes proveedores de cloud (AWS, Azure, GCP), que han captado gran parte del mercado gracias a la agilidad y el modelo operativo que ofrecen.

Parte 2: Arquitectura de Hadoop

Conceptos Fundamentales

Hadoop se basa en un **modelo de despliegue distribuido** sobre un conjunto de servidores que trabajan como una unidad lógica para el usuario. * **Clúster:** Conjunto de servidores que implementan las funcionalidades de Hadoop. * **Nodo:** Cada uno de los servidores individuales que forman parte del clúster.

Tipos de Nodos en la Arquitectura

1. Nodos Master:

- **Función:** Controlan y coordinan el trabajo del clúster. Supervisan la ejecución de tareas (YARN) y el almacenamiento de datos (HDFS).
- **Ejemplo:** Asignan trabajo a los workers, vigilan su estado y reasignan tareas si fallan.

2. Nodos Worker:

- **Función:** Realizan el trabajo pesado. Almacenan los datos y ejecutan las tareas de procesamiento asignadas por los nodos master.

3. Nodos Edge o Frontera:

- **Función:** Actúan como puente entre el clúster y el mundo exterior. Albergan interfaces de cliente, gateways y APIs para que usuarios y aplicaciones externas interactúen con el clúster de forma segura.

Cómo la Arquitectura Logra sus Principales Características

• Alta Disponibilidad / Tolerancia a Fallos:

- **Para Nodos Master:** Se despliegan en configuración **activo-pasivo**. El nodo pasivo mantiene una réplica sincronizada del estado del activo. Si el activo falla, el pasivo toma el control sin interrupción del servicio.

- **Para Nodos Worker (Almacenamiento):** HDFS replica cada bloque de datos (por defecto, 3 veces) en distintos nodos workers. Si un nodo o disco falla, los datos siguen disponibles desde sus réplicas.
 - **Para Nodos Worker (Procesamiento):** El nodo maestro (ResourceManager de YARN) monitoriza constantemente el estado de los workers. Si un worker falla durante una tarea, el maestro reasigna esa tarea a otro worker disponible.
 - **Para Nodos Edge:** Se suelen desplegar varios para redundancia, aunque su criticidad es menor.
- **Escalabilidad Lineal:**
 - Para aumentar la capacidad (de almacenamiento o procesamiento), basta con **añadir nuevos nodos worker** al clúster. El nodo maestro los detecta y redistribuye la carga de trabajo de forma automática o mediante un rebalanceo. El proceso inverso (reducir capacidad) es igualmente posible.
- **Hardware No Específico (Commodity):**
 - Hadoop está diseñado para ejecutarse en hardware estándar, no en costosos servidores de alta gama.
 - **Diferenciación:** Los **nodos master** requieren hardware más confiable (más RAM, discos en RAID, fuentes de alimentación redundantes) debido a su criticidad. Los **nodos worker** pueden usar hardware más básico, ya que los fallos se mitigan a nivel de software (replicación, reejecución de tareas).

Ejemplo Práctico de Caso de Uso

Objetivo: Traer datos de una base de datos Oracle, almacenarlos y hacer consultas SQL para calcular métricas. **Combinación de componentes correcta: HDFS + YARN + Sqoop + Hive**

- * **HDFS:** Para almacenar los datos de forma distribuida y tolerante a fallos.
- * **YARN:** Para gestionar los recursos del clúster y orquestar la ejecución de las tareas.
- * **Sqoop:** Para importar los datos estructurados desde la base de datos Oracle hacia HDFS.
- * **Hive:** Para realizar consultas tipo SQL (HiveQL) sobre los datos almacenados en HDFS y calcular las métricas (medias, máximos, etc.).

Resumen Detallado: Arquitectura de Hardware, Costes y Casos de Uso de Hadoop

1. Arquitectura de Hardware en un Clúster Hadoop

Nodos Maestros (Master)

- **Función:** Ejecutan servicios críticos de coordinación y gestión del clúster (como el NameNode y ResourceManager en HDFS/YARN). No almacenan datos de usuario.
- **Características de Hardware:**
 - **Almacenamiento:** 2-4 discos en RAID (1, 10 o 5) con 2-4 TB de capacidad. Se prioriza la redundancia.
 - **CPU:** 2 CPUs, cada una con 6-8 núcleos. Crítico por la intensidad computacional de los servicios.
 - **Memoria:** 128-256 GB de RAM de alta calidad.
 - **Red:** Conexión de alto rendimiento (10-20 Gbps, incluso Infiniband >50 Gbps). Es un elemento crítico para evitar cuellos de botella.
 - **Fuente de Alimentación:** Redundante para garantizar la disponibilidad.
 - **Consideraciones:** Su potencia debe escalar con el tamaño del clúster. Su fallo no detiene el servicio inmediatamente, pero su recuperación es compleja.

Nodos Trabajadores (Worker)

- **Función:** Almacenan datos (HDFS) y ejecutan tareas de procesamiento (MapReduce, Spark, etc.). Se asume que pueden fallar.
- **Características de Hardware:**
 - **Almacenamiento:** Múltiples discos (10-12) en configuración JBOD (Just a Bunch of Disks), de 3-4 TB cada uno. La replicación se gestiona a nivel de software (HDFS).

- **CPU:** 2 CPUs de gama media con 6-8 núcleos cada una.
- **Memoria:** 64-256 GB de RAM.
- **Red:** Similar a los nodos maestros (10-20 Gbps).
- **Fuente de Alimentación:** No suele ser redundante. La inversión se prioriza en capacidad de cómputo y almacenamiento.
- **Filosofía:** Se invierte en capacidad bruta (almacenamiento/CPU) más que en resiliencia hardware, ya que la tolerancia a fallos es responsabilidad del software.

Nodos Frontera (Edge)

- **Función:** Actúan como punto de entrada para clientes y servicios. Su configuración es similar a la de los nodos maestros.

Coste Aproximado del Hardware (por nodo)

- **Nodo Maestro:** 5.000 - 15.000 €
- **Nodo Worker:** 3.000 - 12.000 €
- **Nodo Edge:** 5.000 - 10.000 €

Ejemplo de clúster: 50 workers + 4 masters + 2 edge. - **Coste hardware total:** ~430.000 €. - **Capacidad de almacenamiento:** ~400 TB.

2. Modelo de Costes Total de Hadoop

1. **Coste de Hardware:** Compra de servidores y equipos de red.
2. **Coste de Soporte Empresarial:** Para distribuciones comerciales (Cloudera, Hortonworks), entre 5.000 y 15.000 € por nodo/año. En el ejemplo, ~350.000 €/año.
3. **Coste de Consultoría/Implementación:** Depende de la complejidad del proyecto.

Comparativa: Un clúster Hadoop sigue siendo significativamente más económico que soluciones tradicionales como bases de datos relacionales de gama alta, que pueden superar 1-2 millones de € anuales solo en licencias.

3. Beneficios, Desventajas y Casos de Uso de Hadoop

Hadoop como Plataforma Integral

Hadoop no es la mejor herramienta para cada caso de uso específico, pero es la **plataforma más versátil** que cubre la mayoría de ellos con buen rendimiento, evitando la necesidad de múltiples herramientas especializadas. - **Ejemplos de alternativas específicas:** - **Tiempo real/Acceso rápido:** Apache Cassandra. - **Cuadros de mando/OLAP:** SAP HANA. - **Transformación de datos (ETL):** Herramientas ETL tradicionales. - **Machine Learning:** Lenguaje R.

Resumen de Características Clave (Beneficios)

1. **Almacena y procesa cualquier tipo de dato:** Estructurados, semi-estructurados y no estructurados.
2. **Schema-on-Read:** Los datos se almacenan en crudo (raw data) y el esquema se aplica al leerlos. Esto permite gran flexibilidad y rapidez para incorporar datos de estructuras variables.
3. **Bajo coste:** Utiliza hardware commodity y software de código abierto.
4. **Escalabilidad lineal y "casi ilimitada":** Hasta ~10,000 nodos (50-100 PB) en un solo clúster.
5. **Enfoque distribuido:** Excelente rendimiento para tareas complejas y volúmenes masivos de datos.
6. **Ecosistema de herramientas:** Ofrece una base para construir aplicaciones y múltiples herramientas (Hive, Spark, HBase, etc.) para casos de uso concretos.

Problemáticas y Desventajas

1. **Falta de profesionales cualificados:** Perfiles escasos y demandados, lo que incrementa costes salariales y riesgo de rotación.
2. **Costes realistas:** "Hardware commodity" no significa hardware obsoleto, y "open-source" no implica coste cero (soporte, mantenimiento).
3. **Complejidad de integración:** Requiere formación del personal e integración en arquitecturas de datos existentes, lo cual no es trivial.

4. **Complejidad administrativa:** Gestionar N nodos con M componentes es más complejo que un sistema centralizado.
5. **Ineficiente para cargas pequeñas o simples:** No es la mejor opción para consultas puntuales o volúmenes de datos bajos.
6. **Madurez en seguridad y gobierno de datos:** Menor que en bases de datos relacionales con décadas de desarrollo.

¿Cuándo USAR Hadoop?

- Volumen de datos que excede la capacidad de sistemas tradicionales.
- Variedad de datos (formatos diversos o cambiantes).
- Necesidad de escalabilidad masiva en almacenamiento o procesamiento.
- Cuando se busca una **plataforma única** para cubrir múltiples casos de uso analíticos con datos masivos.

¿Cuándo NO USAR Hadoop?

- Cuando los sistemas tradicionales (bases de datos relacionales) pueden manejar el volumen y la complejidad de los datos.
- Cuando los formatos de datos son fijos y estables.
- Para **casos de uso operacionales con alta transaccionalidad** (ej.: sistemas de pagos en tiempo real en un banco).
- Para un único caso de uso muy específico que otra herramienta resuelva mejor.

Ejercicio Resuelto (Contexto Bancario): El problema descrito (datos fragmentados por canales y gran volumen de clicks web) es un **caso de uso ideal para Hadoop**. - **Solución con Hadoop:** Se podría crear un **Data Lake** centralizado en Hadoop para ingerir y almacenar todos los datos crudos (raw data) de los diferentes canales (llamadas, visitas a oficina, clicks web, etc.). - **Beneficios:** 1. **Unificación:** Un director de oficina podría consultar un perfil unificado del cliente que incluya la reclamación telefónica. 2. **Procesamiento de Volumen:** Los logs de clicks web, masivos, se podrían almacenar y procesar para análisis de comportamiento. 3. **Schema-on-Read:** Se pueden incorporar nuevos tipos de datos de cualquier canal sin redefinir esquemas complejos previamente. - **Hadoop no reemplazaría** los sistemas transaccionales críticos del banco, sino que **los complementaría**, consolidando los datos para análisis y una visión 360º del cliente.

Resumen: Aplicación de Hadoop en Entornos Empresariales y Guía de Instalación

Sección 1: Casos de Uso de Hadoop en el Sector Bancario

- **Problema Actual:** Los bancos no comparten información del cliente entre canales (oficinas, online, teléfono) debido a limitaciones técnicas y al gran volumen de datos (ej: pagos con tarjeta, llamadas, emails).
- **Solución con Hadoop:**
- **Ficha Única de Cliente:** Almacenar datos de todos los canales para tener una visión unificada. Ejemplo: Un gestor en oficina podría ver la reclamación que un cliente hizo por teléfono el día anterior y resolverla, mejorando la satisfacción.
- **Análisis de Datos No Estructurados:** Procesar emails y transcripciones de llamadas para identificar motivos de quejas, predecir picos de reclamaciones y prescribir acciones correctivas.
- **Modelos Predictivos:** Combinar toda la información para predecir:
 - Propensión a contratar productos.
 - Riesgo de fuga de clientes.
- **Casos de Uso Avanzados:**
 - Decidir ubicación de cajeros analizando pagos con tarjeta y extracciones en cajeros de la competencia.
 - Evaluar riesgo de un comercio según el perfil de clientes que pagan allí (renta, saldo, etc.).

Conclusión: Hadoop permite abordar casos de uso que con tecnologías tradicionales serían inviables o muy costosos, lo que explica por qué los bancos fueron pioneros en su adopción masiva.

⚖ Sección 2: Evaluación de la Necesidad de Hadoop - Ejercicio Práctico

Escenario: Una aseguradora con 300.000 clientes, datos estructurados (contacto y pólizas), que quiere cuadros de mando y modelos predictivos.

Análisis y Respuestas a Afirmaciones Verdadero/Falso: 1. "Hadoop no es probablemente la mejor tecnología para cada caso de uso concreto, pero es bastante buena para la mayoría." → **FALSO**. Hadoop es excelente para problemas de gran volumen y variedad, pero no es la mejor opción para todos los escenarios. 2. "Hadoop es bastante eficiente incluso con pocos datos." → **FALSO**. Al ser una tecnología distribuida, es poco eficiente para volúmenes pequeños (ej: 300.000 clientes). Una base de datos relacional convencional es más adecuada en este caso. 3. "Si no sé qué tecnología Big Data implantar... Hadoop puede ser una buena opción." → **VERDADERO**. Si hay una gran variedad de casos de uso por cubrir, Hadoop, al ser una plataforma amplia y flexible, puede ser una elección segura y difícil de equivocar.

Conclusión del Ejercicio: Para la aseguradora descrita, **NO sería beneficioso** desplegar Hadoop porque: - El volumen de datos es manejable por bases de datos relacionales tradicionales. - Los datos son solo estructurados. - Los casos de uso (cuadros de mando y modelos predictivos básicos) se pueden resolver con herramientas de visualización y machine learning estándar, sin necesidad de capacidades Big Data.

Sección 3: Guía de Instalación de Apache Hadoop con Docker

Objetivo: Aprender a instalar Apache Hadoop y herramientas de su ecosistema en un equipo personal.

Metodología: Uso de contenedores Docker para simplificar la instalación y configuración. - **Recursos Proporcionados:** Archivo comprimido `guia_BDA01.zip` que contiene: - Guía en formato `HTML` (para abrir en navegador). - Guía en formato `IPYNB` (para abrir en Jupyter Notebook). - Vídeo tutorial paso a paso. - Archivo con enlaces de interés. - **Contenido de la Guía:** 1. Instalación de Docker (con explicación básica para principiantes). 2. Configuración de contenedores con Hadoop. 3. Uso de Python en Jupyter Notebook, preparando el entorno para

programar tareas MapReduce en futuras unidades. - **Requisito Previo:** Haber leído los apartados teóricos anteriores de la unidad. - **Soporte:** Se recomienda usar el foro para resolver dudas o problemas durante la instalación.

Propósito: Esta instalación práctica facilitará el desarrollo y la experimentación con programas MapReduce en Hadoop en las siguientes unidades del curso.